Heaven's light is our guide

Rajshahi University of Engineering &Technology

Department of Computer Science & Engineering



Lab Report

Title	**:** Reflection Essay on a Full Software Project
Lab No.	**:** 05
Course Name	**:** Software Engineering Sessional
Course Code	**:** CSE 3206
Submission Date	**:** 31 October, 2026

Submitted by:	**Submitted to:**
Name: Md. Mhafuz Taraq Farazi **Roll**: 2203031 **Section:** A **Session:** 2022-2023	**Name:** Emrana Kabir Hashi **Designation:** Assistant Professor **Department:** CSE **Institution:** RUET

LearnTube: YouTube Coding Tutorial Learning Platform

Implementation language: Python | Process model: Kanban | Testing: pytest

Problem and Scope

Many learners save coding videos without a structured way to discover, organize, and complete them. LearnTube extracts a safe YouTube embed link from a tutorial URL and combines a global catalog, search, learner accounts, enrollment, progress tracking, notes, and a private dashboard for each user. SQLite provides persistent local storage. The application is designed as a domain-service package so a future web interface can reuse the same business rules.

Requirements

Functional requirements:

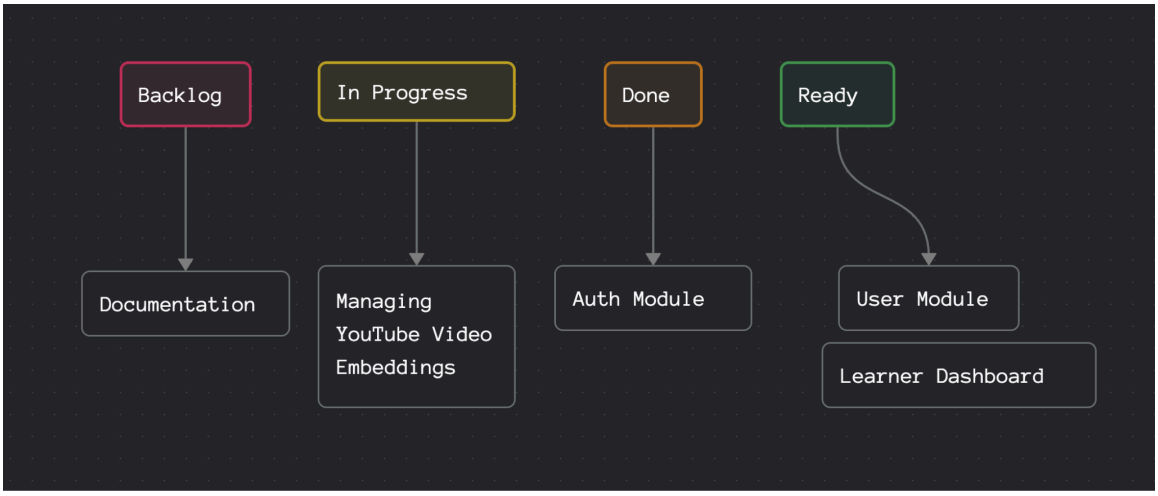
1. FR1: a learner can register with a name and email.
2. FR2: an administrator or catalog editor can add a tutorial with title, description, topic, level, and YouTube URL.
3. FR3: the system validates the URL and stores the embed link.
4. FR4: a learner can search the global catalog by keyword or level.
5. FR5: an authenticated learner can enroll in a tutorial.
6. FR6: a learner can update progress from 0 to 100.
7. FR7: a learner can save notes; and (FR8) each learner has a private dashboard showing their enrolled tutorials and progress.

Non-functional requirements: input validation must reject unsafe or incomplete data; passwords must not be stored in plain text; data must survive process restarts through SQLite; the service must be testable without a network connection; modules must have single responsibilities; and common operations should respond quickly for a small academic catalog.

User stories: As a visitor, I want to find tutorials by topic so that I can choose a relevant lesson. As an authenticated learner, I want to enroll from the global catalog and see only my own dashboard. As a learner, I want to record progress and notes so that I can continue later. As a catalog editor, I want invalid YouTube links rejected so that every saved tutorial can be embedded.

Kanban Process Model

Kanban was selected because the project is small but has changing requirements and benefits from visible work-in-progress. The board used four columns: Backlog, Ready, In Progress, and Done. Work items were limited to one



active implementation item at a time. Initial cards were: define user stories; design SQLite schema; implement YouTube provider; implement catalog and users; implement enrollment and notes; write tests; document patterns; and prepare the report. A card moved only after its acceptance condition and relevant tests were complete. This model was preferred over Waterfall because validation feedback from tests could change the next card without waiting for a full phase.

Design Patterns

Factory/Provider abstraction (creational): VideoProvider lets the service create or inject a YouTube integration without coupling business logic to network code. The rejected alternative was constructing a network client directly inside add_tutorial, which would make tests slow and fragile.

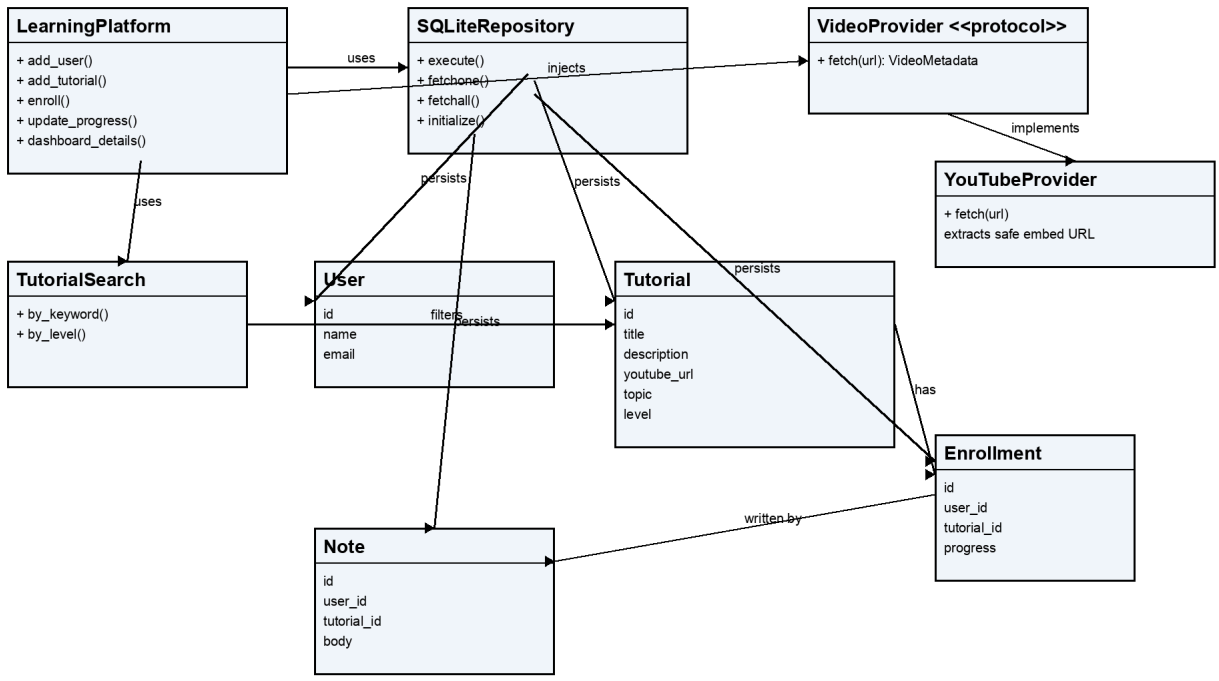
Repository (structural): SQLiteRepository hides SQL connection and persistence details behind execute, fetchone, and fetchall. The rejected alternative was placing SQL in every service method, which duplicates transaction and connection handling.

Strategy (behavioral): TutorialSearch separates keyword and level filtering strategies and can be extended with duration or recommendation strategies. The rejected alternative was a single large conditional search function, which would become difficult to extend.

UML Class Diagram

The UML class diagram below shows the main application service, persistence repository, YouTube provider abstraction, search component, and persistent domain entities. Arrows indicate service usage, implementation, filtering, and stored relationships.

LearnTube UML Class Diagram



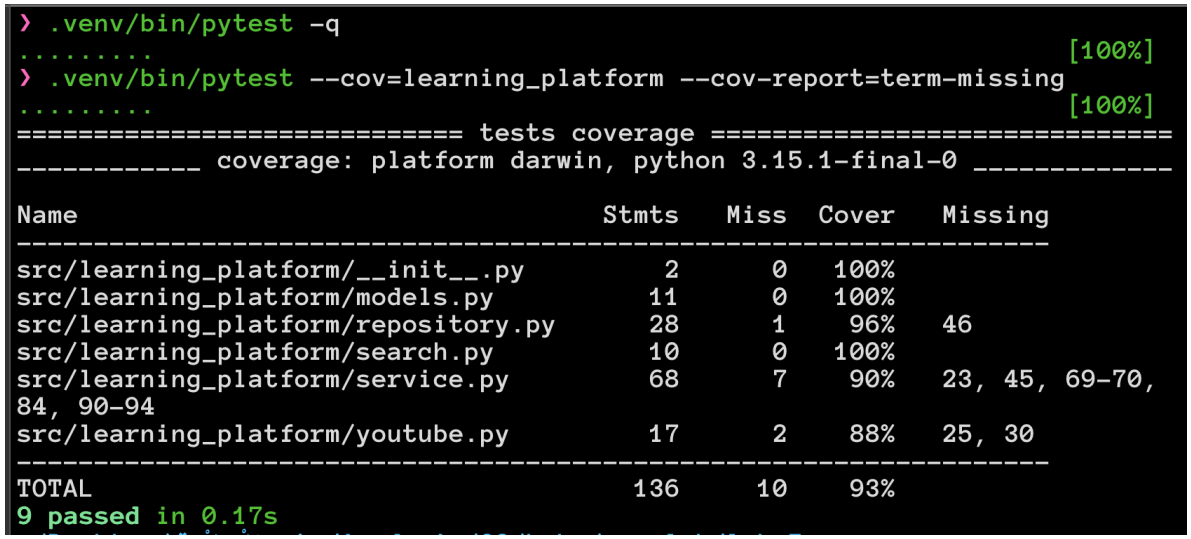
pytest Test Report

```
› .venv/bin/pytest --collect-only
tests/test_platform.py::test_add_tutorial_uses_provider_and_persists_embed_link
tests/test_platform.py::test_enrollment_and_progress_are_recorded
tests/test_platform.py::test_invalid_progress_is_rejected
tests/test_platform.py::test_empty_note_is_rejected
tests/test_platform.py::test_user_authentication_requires_correct_password
tests/test_platform.py::test_short_password_is_rejected
tests/test_platform.py::test_dashboard_contains_only_the_authenticated_users_enrollments
tests/test_platform.py::test_tutorials_can_be_searched_by_keyword_and_level
tests/test_platform.py::test_youtube_provider_rejects_non_youtube_url

9 tests collected in 0.01s
```

Test command executed: .venv/bin/pytest --cov=learning_platform --cov-report=term-missing. Result: 9 tests passed, 0 failed, in 0.19 seconds. The tests use an in-memory SQLite repository and a FakeVideoProvider mock, so the result is repeatable and independent of network availability.

Test area	What was verified	Result
Tutorial persistence	YouTube URL is converted to an embed link and stored.	Passed
Enrollment and progress	A learner enrolls and updates progress to 60%.	Passed
Validation	Invalid progress and empty notes are rejected.	Passed
Authentication	Correct credentials succeed; incorrect credentials fail.	Passed
Password policy	Passwords shorter than eight characters are rejected.	Passed
Dashboard isolation	Each learner sees only their own enrollments.	Passed
Catalog search	Keyword and level filters return matching tutorials.	Passed
YouTube validation	Non-YouTube URLs are rejected.	Passed
Module	**Statements**	**Coverage**
repository.py	28	96%
search.py	10	100%
service.py	68	90%
youtube.py	17	88%
Overall	136	93%



Coverage summary from the executed report: 136 statements measured, 126 covered, for 93% total coverage. The remaining uncovered lines are defensive error branches and provider URL edge cases; the passing tests cover the main user workflows and validation rules.

New Tool Learned (pytest, AI-Assisted Development & Github Releases)

pytest was selected as the self-directed tool because it provides readable assertions, fixtures, parametrization, and plugin-based coverage with little boilerplate. I applied fixtures for an isolated repository, a fake external provider for mocking, exception assertions, and pytest-cov for measurement. The alternative was unittest, which is part of the standard library but would require more class-based setup and less concise exception checks. pytest was more suitable for rapid Kanban feedback and TDD.

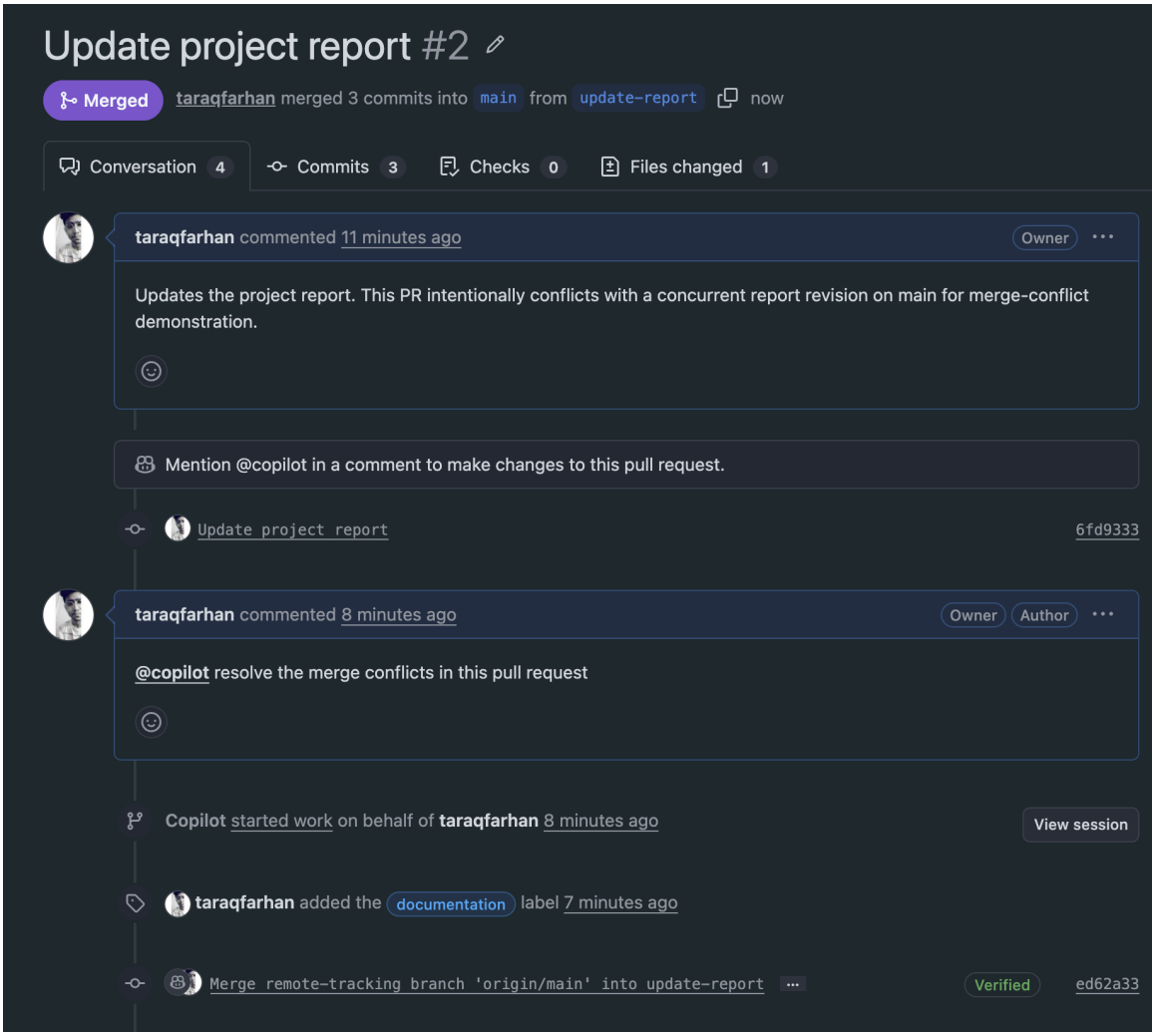
AI-assisted development with GitHub Copilot was applied as a support tool during implementation and review. It helped explain unfamiliar Python behavior, suggest small implementation alternatives, identify edge cases, and improve documentation. I remained responsible for checking every suggestion, adapting it to the project architecture, and running pytest. This tool was chosen instead of relying only on manual searching because it provides context-aware assistance inside the coding workflow; however, generated suggestions were not accepted without understanding and verification.

GitHub Releases was also learned and applied as a version-distribution practice. A release can identify a stable LearnTube version, summarize its features, link the project report, and provide a clear milestone for demonstration. This is different from ordinary commits because it communicates a usable project snapshot to teachers and users. The alternative was using only branch names or commit history, which does not provide a simple stable version for evaluation.

Together, Copilot and GitHub Releases extend pytest: Copilot supports development quality, pytest verifies behavior, and Releases communicate validated deliverables.

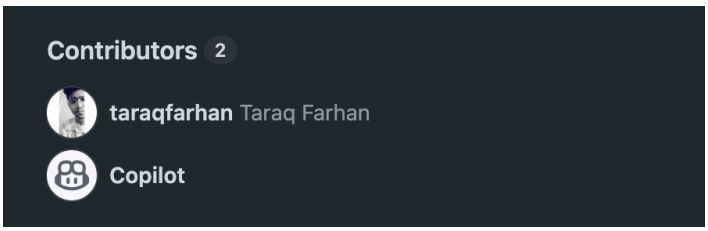
Git/GitHub Evidence

Pull Request and Resolving Merge Conflict with Github Copilot



Merge-conflict resolution evidence: merged origin/main into update-report, resolved the binary conflict in docs/LearnTube_Project_Report.docx by keeping the PR branch report version, and finalized merge commit ed62a33.

Github Copilot Contribution



Git History

```
❯ git log --graph --all --oneline --decorate=no
*   e418de1 Merge pull request #2 from taraqfarhan/update-report
|\
| * 7caf3a9 Document merge conflict resolution evidence
| *   ed62a33 Merge remote-tracking branch 'origin/main' into update-report
| |\
| |/
|/|
* | 96da4e6 Revise project report
| * 6fd9333 Update project report
|/
*   2af56b2 Merge pull request #1 from taraqfarhan/docs/readme-update
|\
| * b2ed1b6 Updating README
|/
* 04830ad Ignore local report generator
* 6a2bd64 Add project report
* c7d62d9 Document project usage
* e7bb8d2 Add web application
* 3ab90eb Add command line demonstration
* 5f32af7 Add pytest coverage
* 0b8f847 Add learning platform services
* 1fb8917 Add tutorial search
* 3f1552f Add YouTube provider
* cfaf6c6 Add SQLite repository
* 0a05fbc Add learning domain models
* 5cb70c0 Add project configuration
```

Git Branches

```
❯ git branch --all
  docs/readme-update
* main
  update-report
  remotes/origin/HEAD -> origin/main
  remotes/origin/docs/readme-update
  remotes/origin/main
  remotes/origin/update-report
```

Learning Log and Reflection Essay

I chose a YouTube-based learning platform because it connects a familiar learner problem with important software engineering concerns: external input validation, persistent data, modular services, authentication, and measurable progress. My main technical decision was to separate the domain service from the browser interface. LearningPlatform coordinates users, tutorials, enrollment, progress, and notes; SQLiteRepository owns persistence; YouTubeProvider validates links and creates safe embed URLs; and the standard-library web application renders the user experience. This structure means that a future framework or mobile client can reuse the same business rules.

The first learning-log milestone was requirements analysis. I converted the open-ended idea into a global catalog, private user dashboards, search, enrollment, progress, notes, and embedded videos. The second milestone was persistence: I selected SQLite because it is reliable for a course prototype, requires no separate server, and makes the data survive application restarts. The third milestone was testing. I used pytest fixtures with an in-memory database and a fake VideoProvider, so tests remain fast and do not depend on YouTube availability. The final milestones were authentication, dashboard authorization, report documentation, and a proper UML diagram.

The most difficult challenge was deciding how much YouTube functionality to implement. A full scraper or transcript service would create API-key, rate-limit, network, and copyright-related complications. I chose a narrower and more dependable solution: accept YouTube URLs, validate the host and video identifier, store the embed link, and display the video inside the application. Another challenge was preventing learners from seeing each other's dashboards. Initially, the page exposed user selection and navigation did not consistently preserve the session. I corrected this by using password hashes, signed HttpOnly session cookies, authenticated enrollment, user-specific dashboard queries, ownership checks for progress and notes, and a logout action that clears the cookie.

For self-directed learning, I studied pytest documentation, fixture examples, assertion examples, and pytest-cov usage. These resources were strong for learning the framework syntax and test organization because they were concise and supported by executable examples. Their limitation was that they did not directly explain how to isolate my custom YouTube boundary. I addressed that gap by applying the examples to a VideoProvider protocol and writing a fake provider. I also consulted Python's sqlite3 and hashlib documentation to understand in-memory databases, thread-safe access, and PBKDF2 password hashing. Official documentation was more reliable than short tutorials, although tutorials were useful for seeing complete workflows quickly.

The project is still a course-scale prototype. A production release would need a stronger web framework, CSRF protection, secure deployment configuration, database migrations, password reset, rate limiting, structured logging, and more comprehensive integration tests. I also learned that a coverage percentage is only a signal; meaningful tests for authorization, invalid input, and user isolation matter more than a number alone.

My next skill will be CI/CD pipelines. I plan to learn GitHub Actions, create a workflow that installs the requirements, runs pytest and coverage on every push and pull request, and blocks a merge when tests fail. I will then add separate jobs for linting and packaging, store coverage artifacts, and configure a deployment step for a staging environment. This follows naturally from LearnTube because the project already has a repeatable test command and a clear separation between source, tests, and documentation. Overall, this work taught me that software quality comes from connecting requirements, design, implementation, testing, security, and continuous improvement rather than treating them as unrelated report sections.

Conclusion

LearnTube demonstrates a complete, extensible software engineering solution for organizing YouTube-based coding tutorials. The project connects requirements, Kanban planning, modular design, design patterns, persistent SQLite storage, embedded videos, secure authentication, private learner dashboards, enrollment, progress tracking, notes, and pytest-based validation. The global catalog makes learning resources easy to discover, while user-specific dashboards protect each learner's progress and personal notes. The project also shows the value of testable architecture through the repository abstraction and mocked YouTube provider. Although future improvements such as production deployment, stronger web security, password recovery, recommendations, and CI/CD automation are planned, the current implementation satisfies the core learning-platform goals and provides a sound foundation for those extensions.